
Online normalizer calculation for softmax

Maxim Milakov
NVIDIA
mmilakov@nvidia.com

Natalia Gimelshein
NVIDIA
ngimelshein@nvidia.com

Abstract

The Softmax function is ubiquitous in machine learning, multiple previous works suggested faster alternatives for it. In this paper we propose a way to compute classical Softmax with fewer memory accesses and hypothesize that this reduction in memory accesses should improve Softmax performance on actual hardware. The benchmarks confirm this hypothesis: Softmax accelerates by up to 1.3x and Softmax+TopK combined by up to 5x.

1 Introduction

Neural networks models are widely used for language modeling, for tasks such as machine translation [1] and speech recognition [2]. These models compute word probabilities taking into account the already generated part of the sequence. The probabilities are usually computed by a Projection layer, which "projects" hidden representation into the output vocabulary space, and a following Softmax function, which transforms raw logits into the the vector of probabilities.

A number of previous works suggested faster alternatives to compute word probabilities. Differentiated Softmax [3] and SVD-Softmax [4] replace the projection layer - which is usually just a matrix multiplication - with more computationally efficient alternatives. Multiple variants of Hierarchical Softmax [5, 6, 7] split a single Projection+Softmax pair into multiple much smaller versions of these two functions organized in tree-like structures. Sampled-based approximations, such as Importance Sampling [8], Noise Contrastive Estimation [9], and Blackout [10] accelerate training by running Softmax on select elements of the original vector. Finally, Self-Normalized Softmax [11] augments the objective function to make the softmax normalization term close to 1 (and skip computing it during inference).

This is not an exhaustive list, but, hopefully, a representative one. Interestingly, almost all of the approaches still need to run the original Softmax function, either on full vector or reduced one. There are two exceptions that don't need to compute the softmax normalization term: training with Noise Contrastive Estimation and inference with Self-Normalized Softmax. All others will benefit from the original Softmax running faster.

To the best of our knowledge there has been no targeted efforts to improve the performance of the original Softmax function. We tried to address this shortcoming and figured out a way to compute Softmax with fewer memory accesses. We benchmarked it to see if those reductions in memory accesses translate into performance improvements on a real hardware.

2 Original softmax

Function $y = \text{Softmax}(x)$ is defined as:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^V e^{x_j}} \quad (1)$$

where $x, y \in \mathbb{R}^V$. The naive implementation (see algorithm 1) scans the input vector two times - one to calculate the normalization term d_V and another to compute output values y_i - effectively doing three memory accesses per vector element: two loads and one store.

Algorithm 1 Naive softmax

```

1:  $d_0 \leftarrow 0$ 
2: for  $j \leftarrow 1, V$  do
3:    $d_j \leftarrow d_{j-1} + e^{x_j}$ 
4: end for
5: for  $i \leftarrow 1, V$  do
6:    $y_i \leftarrow \frac{e^{x_i}}{d_V}$ 
7: end for

```

Unfortunately, on real hardware, where the range of numbers represented is limited, the line 3 of the algorithm 1 can overflow or underflow due to the exponent. There is a safe form of (1), which is immune to this problem:

$$y_i = \frac{e^{x_i - \max_{k=1}^V x_k}}{\sum_{j=1}^V e^{x_j - \max_{k=1}^V x_k}} \quad (2)$$

All major DL frameworks are using this safe version for the Softmax computation: TensorFlow

Algorithm 2 Safe softmax

```

1:  $m_0 \leftarrow -\infty$ 
2: for  $k \leftarrow 1, V$  do
3:    $m_k \leftarrow \max(m_{k-1}, x_k)$ 
4: end for
5:  $d_0 \leftarrow 0$ 
6: for  $j \leftarrow 1, V$  do
7:    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$ 
8: end for
9: for  $i \leftarrow 1, V$  do
10:   $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
11: end for

```

[12] v1.7, PyTorch [13] (with Caffe2) v0.4.0, MXNET [14] v1.1.0, Microsoft Cognitive Toolkit [15] v2.5.1, and Chainer [16] v5.0.0a1. But Safe Softmax does three passes over input vector: The first one calculates the maximum value m_V , the second one - normalization term d_V , and the third one - final values y_i , see algorithm 2; This results in 4 memory access per vector element overall. We want to improve on that.

3 Online normalizer calculation

The algorithm 3 calculates both the maximum value m and the normalization term d in a single pass over input vector with negligible additional cost of two operations per vector element. It reduces memory accesses from 4 down to 3 per vector element for the Softmax function evaluation. Inspiration came from the numerically stable variance calculation online algorithm, see [17].

Essentially, the algorithm keeps the maximum value m and the normalization term d as it iterates over elements of the input array. At each iteration it needs to adjust the normalizer d to the new maximum m_j and only then add new value to the normalizer.

Theorem 1. *The lines 1-6 of the algorithm 3 compute $m_V = \max_{k=1}^V x_k$ and $d_V = \sum_{j=1}^V e^{x_j - m_V}$*

Proof. We will use a proof by induction.

Algorithm 3 Safe softmax with online normalizer calculation

```

1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3: for  $j \leftarrow 1, V$  do
4:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
5:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$ 
6: end for
7: for  $i \leftarrow 1, V$  do
8:    $y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$ 
9: end for

```

◇ *Base case:* $V = 1$

$$\begin{aligned}
m_1 &\leftarrow x_1 && \text{by line 4 of the algorithm 3} \\
&= \max_{k=1}^1 x_k \\
d_1 &\leftarrow e^{x_1-m_1} && \text{by line 5 of the algorithm 3} \\
&= \sum_{j=1}^1 e^{x_j-m_1}
\end{aligned}$$

The theorem holds for $V = 1$.

◇ *Inductive step:* We assume the theorem statement holds for $V = S - 1$, that is the lines 1-6 of the algorithm 3 compute $m_{S-1} = \max_{k=1}^{S-1} x_k$ and $d_{S-1} = \sum_{j=1}^{S-1} e^{x_j-m_{S-1}}$. Let's see what the algorithm computes for $V = S$

$$\begin{aligned}
m_S &\leftarrow \max(m_{S-1}, x_S) && \text{by line 4 of the algorithm 3} \\
&= \max(\max_{k=1}^{S-1} x_k, x_S) && \text{by the inductive hypothesis} \\
&= \max_{k=1}^S x_k \\
d_S &\leftarrow d_{S-1} \times e^{m_{S-1}-m_S} + e^{x_S-m_S} && \text{by line 5 of the algorithm 3} \\
&= \left(\sum_{j=1}^{S-1} e^{x_j-m_{S-1}} \right) \times e^{m_{S-1}-m_S} + e^{x_S-m_S} && \text{by the inductive hypothesis} \\
&= \sum_{j=1}^{S-1} e^{x_j-m_S} + e^{x_S-m_S} \\
&= \sum_{j=1}^S e^{x_j-m_S}
\end{aligned}$$

The inductive step holds as well. □

The algorithm 3 is proved to compute the Softmax function as defined in (2). It is also safe:

- m_j is the running maximum, $m_j \in \left[\min_{k=1}^V m_k, \max_{k=1}^V m_k \right]$, $\forall j \in 1, V$; m_j cannot underflow or overflow.
- d_j is also bounded: $1 \leq d_j \leq j, \forall j \in 1, V$. It can be easily proven by induction. The 32-bit floating point storage for d_j guarantees processing of up to $1.7 * 10^{37}$ elements in vector x without overflow. It is a reasonably large amount, but if your vector is even larger you need to use the 64-bit floating point storage for d_j .

Interestingly, the algorithm 2 provides the same guarantees: $1 \leq d_j \leq j, \forall j \in 1, V$.

In the remainder of this paper we will call algorithm 3 "Online Softmax".

3.1 Parallel online normalizer calculation

The lines 1-6 of the algorithm 3 define a sequential way of calculating the normalization term in a single pass over input vector. Modern computing devices allow running multiple threads concurrently; We need to have a parallel version of the algorithm to fully utilize devices. We define a generalized version of the online normalizer calculation:

$$\begin{bmatrix} m_V \\ d_V \end{bmatrix} = \begin{bmatrix} x_1 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} x_2 \\ 1 \end{bmatrix} \oplus \dots \oplus \begin{bmatrix} x_V \\ 1 \end{bmatrix} \quad (3)$$

where $x_i, m_V, d_V \in \mathbb{R}$. The binary operation $\oplus : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}^2$ is defined as:

$$\begin{bmatrix} m_i \\ d_i \end{bmatrix} \oplus \begin{bmatrix} m_j \\ d_j \end{bmatrix} = \begin{bmatrix} \max(m_i, m_j) \\ d_i \times e^{m_i - \max(m_i, m_j)} + d_j \times e^{m_j - \max(m_i, m_j)} \end{bmatrix} \quad (4)$$

Applying (3) sequentially from left to right is equivalent to running lines 1-6 of the algorithm 3. The operation $\oplus$ is associative, which enables parallel evaluation of (3). It is also commutative, which provides the flexibility needed to make parallel implementations more efficient. We omit the proofs for these two statements for brevity.

4 Softmax and top-k fusion

Online Softmax (algorithm 3) does three memory accesses per vector element: one load for the normalizer calculation, one load and one store for computing Softmax function values y_i . Inference with the beam search for auto-regressive models has TopK following Softmax, and this TopK doesn't need to compute all y_i values. This enables even bigger improvements.

The TopK function is producing the vector of K integer indices referencing the largest values in the input vector, along with those values:

$$TopK(y) = (v, z) : v_i = y_{z_i}, v_i \geq y_j, \forall i \in [1, K], \forall j \notin z \quad (5)$$

where $y \in \mathbb{R}^V, z \in \mathbb{Z}^K, v \in \mathbb{R}^K$.

Algorithm 4 Online softmax and top-k

```

1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3:  $u \leftarrow \{-\infty, -\infty, \dots, -\infty\}^T, u \in \mathbb{R}^{K+1}$   $\triangleright$  The 1st  $K$  elems will hold running TopK values
4:  $p \leftarrow \{-1, -1, \dots, -1\}^T, p \in \mathbb{Z}^{K+1}$   $\triangleright$  ... and their indices
5: for  $j \leftarrow 1, V$  do
6:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
7:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$ 
8:    $u_{K+1} \leftarrow x_j$   $\triangleright$  Initialize  $K + 1$  elem with new value from input vector
9:    $p_{K+1} \leftarrow j$   $\triangleright$  ... and its index
10:   $k \leftarrow K$   $\triangleright$  Sort  $u$  in descending order, permuting  $p$  accordingly. The first  $K$  elements are
    already sorted, so we need just a single loop, inserting the last element in the correct position.
11:  while  $k \geq 1$  and  $u_k < u_{k+1}$  do
12:     $\text{swap}(u_k, u_{k+1})$ 
13:     $\text{swap}(p_k, p_{k+1})$ 
14:     $k \leftarrow k - 1$ 
15:  end while
16: end for
17: for  $i \leftarrow 1, K$  do  $\triangleright$  The algorithm stores only  $K$  values and their indices
18:    $v_i \leftarrow \frac{e^{u_i - m_V}}{d_V}$ 
19:    $z_i \leftarrow p_i$ 
20: end for
```

The TopK needs to load each element of the input vector at least once. Running Safe Softmax and the TopK separately requires 5 accesses per input element and 4 accesses if we use Online Softmax

instead of Safe Softmax (but still run them separately, one after another). If we improve on the algorithm 3 and keep not only running values of m and d (when iterating over the input vector), but also the vectors of TopK input values u and their indices p - as in the algorithm 4 - we can run this Softmax+TopK fusion with just one memory access per element of the input vector.

5 Benchmarking

Online normalizer calculation reduces the amount of memory accesses for the Softmax and Softmax+TopK functions. The softmax function has a very low flops per byte ratio; that means the memory bandwidth should be limiting the performance. Memory access improvements should translate into performance improvements, and experiments confirm this.

We implemented a benchmark for GPUs using CUDA C. The benchmark utilizes CUB v1.8.0 for fast parallel reductions. All experiments were run on NVIDIA Tesla V100 16 GB, ECC on, CUDA Toolkit 9.1. Source code of the benchmark will be made available shortly.

5.1 Benchmarking softmax

We benchmarked all 3 Softmax algorithms - Naive, Safe, and Online - on different vector sizes for the batch sizes of 4,000 and 10. The large batch case corresponds to the training or batch inference with enough input vectors to saturate the device and the small batch case corresponds to online inference with too few vectors to occupy the device fully.

For the large batch case (see figure 1) all three algorithms perform similarly up until $V = 1000$ vector size. The NVIDIA Visual Profiler shows that at that point L1 and L2 cache thrashing starts to make all three algorithms limited by the DRAM bandwidth. When this happens Online and Naive algorithms are getting faster than Safe one, quickly achieving $\sim 1.3x$ at $V = 4000$. This is quite close to 1.33x reduction in memory accesses for those algorithms.

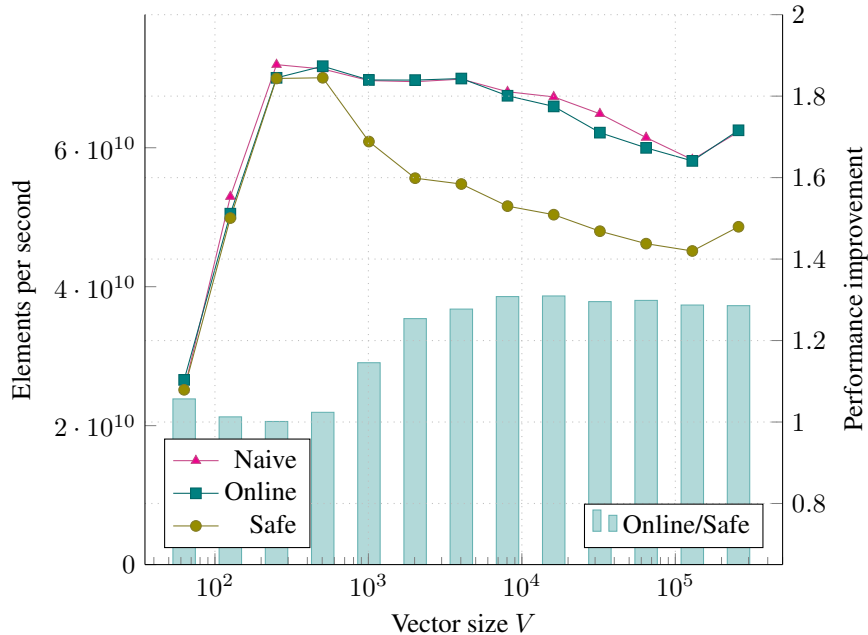


Figure 1: Benchmarking softmax, Tesla V100, fp32, batch size 4000 vectors

The absolute performance for small batch case is lower for all algorithms, see figure 2. The benchmark is running one threadblock per vector; thus small batch case - with 10 vectors - has just 10 threadblocks in the grid. This is not enough to saturate the GPU, both compute and the memory subsystem are underutilized, various latencies are exposed. As in the batch inference case, all three algorithms show similar performance up to $V = 1000$ vector size. After that Naive and Online algorithms outperform Safe one by $\sim 1.15x$.

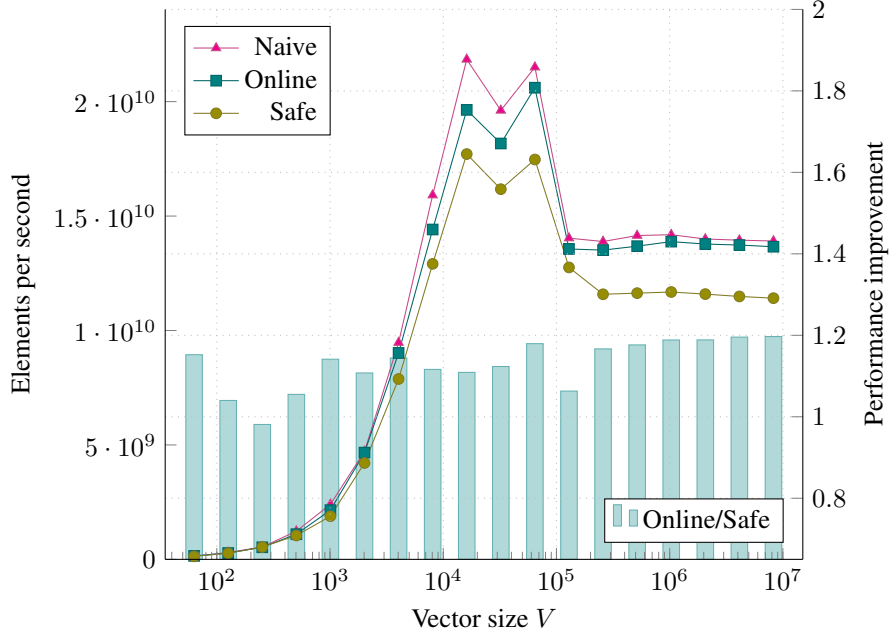


Figure 2: Benchmarking softmax, Tesla V100, fp32, batch size 10 vectors

5.2 Benchmarking softmax and top-k

We benchmarked Safe Softmax followed by the TopK (running one after another) and Online Softmax fused with the TopK into a single function, again, for 2 cases: 4,000 and 10 vectors. We picked up $K = 5$ in TopK for all runs.

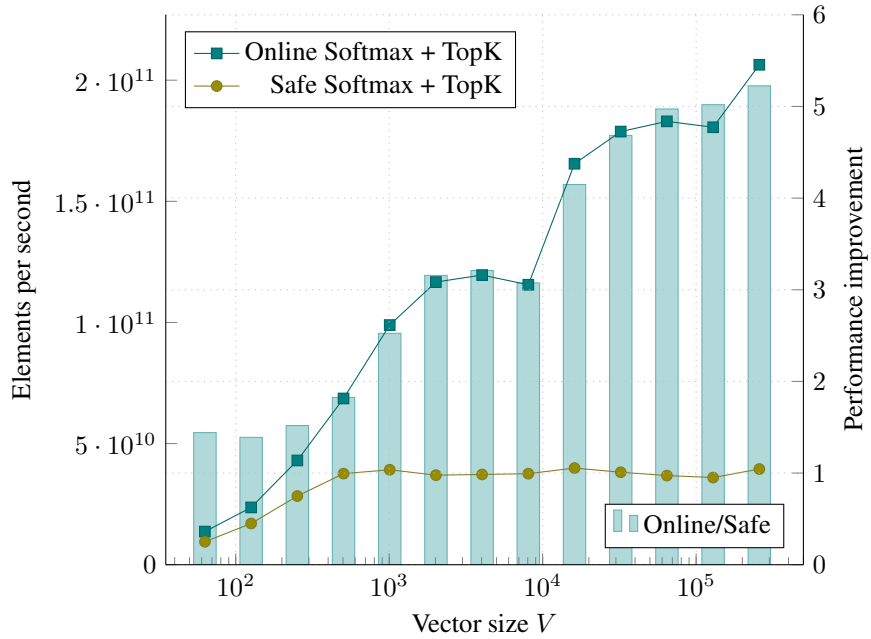


Figure 3: Benchmarking softmax and top-k, Tesla V100, fp32, batch size 4000 vectors

Online fused version is running considerably faster than Safe unfused one. For large batch case - see figure 3 - the performance improvement starts at 1.5x and goes up as vector size V increases approaching 5x at $V = 25000$, which corresponds to 5x reduction in memory accesses.

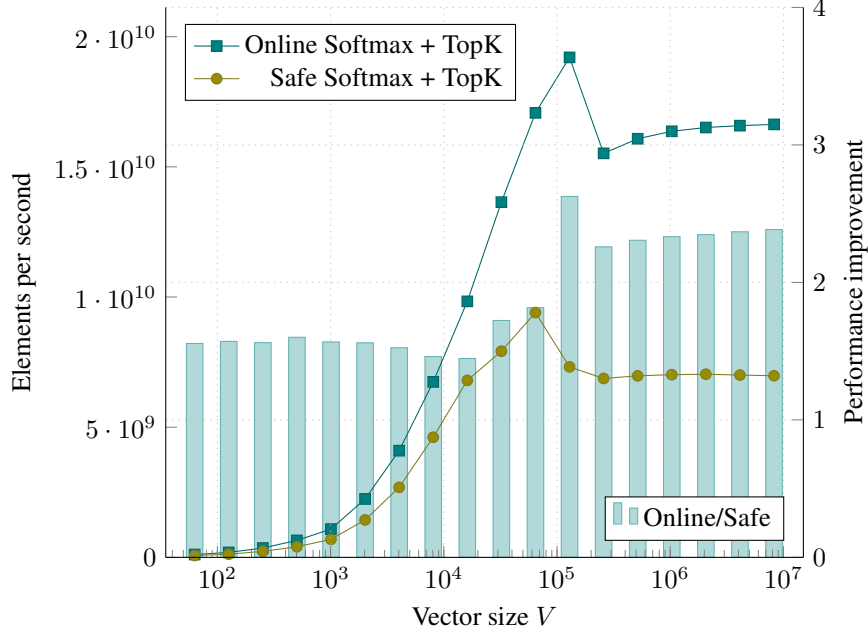


Figure 4: Benchmarking softmax and top-k, Tesla V100, fp32, batch size 10 vectors

For the small batch case (see figure 4) the performance improvement stays between 1.5x and 2.5x. It cannot achieve 5x because the GPU is underutilized and the performance is limited not by the memory bandwidth, but by different latencies. Yet the reduction in memory accesses helps even in this latency limited case.

The benchmark shows these levels of performance improvement for relatively small K only. The cost of keeping partial TopK results - as in the lines 10-15 of the algorithm 4 - increases quickly as K gets bigger: the performance improvement drops to 3.5x for $K = 10$, 2x for $K = 15$, 1.4x for $K = 30$, and degrades further for bigger K s. For these cases the TopK is dominating (in terms of runtime) over the Softmax. Getting rid of separate Softmax and fusing the normalization term calculation into the TopK is still beneficial, but the value goes down as TopK is taking more and more time.

6 Results

We introduced the way to calculate the normalizer for the Softmax function in a single pass over input data, which reduces memory accesses by 1.33x for the Softmax function alone. Benchmarks on Tesla V100 show that this materializes in 1.15x performance improvements for $V \geq 1000$ vector sizes, and for the large batch mode it goes up to 1.3x when $V \geq 4000$.

If one is using Naive Softmax then switching to Online version improves numerical accuracy with no performance hit or a negligible one.

When the TopK follows the Softmax the new single-pass normalizer calculation enables efficient fusion of these 2 functions resulting in 5x fewer memory accesses for Softmax+TopK combined. We observed 1.5x-5x performance improvement on Tesla V100.

These performance improvements could be applied not only to the classical Softmax function; They are orthogonal to many other Softmax optimization techniques including Hierarchical Softmax, Importance Sampling, and SVD-Softmax.

7 Discussion

Online Softmax is running up to 1.3x faster on the latest generation GPU than the one used by major DL frameworks. It also enables very efficient fusion of the Softmax with following TopK showing up to 5x performance improvement over the traditional Safe Softmax and TopK running separately.

Could we see significantly different speed-ups or even slow-downs on different compute devices, for example CPUs? We didn't do experiments for those, but if the original code is vectorized and one manages to keep it vectorized for the online normalizer (and partial TopK) calculation then similar speedups could probably be expected.

Is there a way to improve the performance further? Maybe. The resulting Softmax and even Softmax+TopK fused are still limited by the memory bandwidth. If the preceding function is compute bound then most of Softmax+TopK could probably be fused into that layer. This change is more challenging though.

Acknowledgments

We would like to thank Christoph Angerer for his valuable comments and suggestions.

References

- [1] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to Sequence Learning with Neural Networks. *ArXiv e-prints*, September 2014, 1409.3215.
- [2] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A. r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 29(6):82–97, Nov 2012. ISSN 1053-5888.
- [3] W. Chen, D. Grangier, and M. Auli. Strategies for Training Large Vocabulary Neural Language Models. *ArXiv e-prints*, December 2015, 1512.04906.
- [4] Kyuhong Shim, Minjae Lee, Iksoo Choi, Yoonho Boo, and Wonyong Sung. Svd-softmax: Fast softmax approximation on large vocabulary neural networks. In *Advances in Neural Information Processing Systems 30*, pages 5463–5473. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/7130-svd-softmax-fast-softmax-approximation-on-large-vocabulary->
- [5] Joshua Goodman. Classes for fast maximum entropy training. In *ICASSP*, pages 561–564. IEEE, 2001. ISBN 0-7803-7041-4. URL <http://dblp.uni-trier.de/db/conf/icassp/icassp2001.html#Goodman01>.
- [6] T. Mikolov, S. Kombrink, L. Burget, J. Černocký, and S. Khudanpur. Extensions of recurrent neural network language model. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5528–5531, May 2011.
- [7] E. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou. Efficient softmax approximation for GPUs. *ArXiv e-prints*, September 2016, 1609.04309.
- [8] Yoshua Bengio and Jean-Sébastien S  n  cal. Quick training of probabilistic neural nets by importance sampling. In *Proceedings of the conference on Artificial Intelligence and Statistics (AISTATS)*, 2003.
- [9] A. Mnih and Y. Whye Teh. A Fast and Simple Algorithm for Training Neural Probabilistic Language Models. *ArXiv e-prints*, June 2012, 1206.6426.
- [10] S. Ji, S. V. N. Vishwanathan, N. Satish, M. J. Anderson, and P. Dubey. BlackOut: Speeding up Recurrent Neural Network Language Models With Very Large Vocabularies. *ArXiv e-prints*, November 2015, 1511.06909.
- [11] Jacob Devlin, Rabih Zbib, Zhongqiang Huang, Thomas Lamar, Richard Schwartz, and John Makhoul. Fast and robust neural network joint models for statistical machine translation. In *Proceedings of ACL2014*, pages 1370–1380, 2014.

- [12] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from tensorflow.org.
- [13] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017.
- [14] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Twenty-ninth Annual Conference on Neural Information Processing Systems (NIPS)*, 2015. URL <https://github.com/dmlc/web-data/raw/master/mxnet/paper/mxnet-learningSys.pdf>.
- [15] Frank Seide and Amit Agarwal. Cntk: Microsoft’s open-source deep-learning toolkit. In *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’16, pages 2135–2135, New York, NY, USA, 2016. ACM. ISBN 978-1-4503-4232-2. URL <http://doi.acm.org/10.1145/2939672.2945397>.
- [16] Seiya Tokui, Kenta Oono, Shohei Hido, and Justin Clayton. Chainer: a next-generation open source framework for deep learning. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Twenty-ninth Annual Conference on Neural Information Processing Systems (NIPS)*, 2015. URL http://learningsys.org/papers/LearningSys_2015_paper_33.pdf.
- [17] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. URL <https://amstat.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022>.